LECTURE 13

MODEL REASONING: CAPABILITIES, LIMITATIONS, REASONABLE EXPECTATIONS

DATASCI290: NEUROSYMBOLIC AI

FEBRUARY 23 2026

WHAT IS **HUMAN** REASONING ?

TYPES OF REASONING

- **Deductive reasoning:** start from general rules or premises and derive conclusions that must be true if the premises are true.
- **Inductive reasoning:** generalize from examples/observations to a broader rule; conclusions are probable, not guaranteed.
- **Abductive reasoning:** infer the most plausible explanation for incomplete or noisy observations ("best explanation").
- **Analogical reasoning:** transfer structure from a known situation to a new one by mapping correspondences.
- **Causal reasoning:** reason about cause–effect relations; predict consequences of interventions and counterfactuals ("what if we change X?").
- **Probabilistic/statistical reasoning:** update beliefs under uncertainty using likelihoods/base rates (explicitly or implicitly).
- **Practical reasoning / planning:** choose actions to achieve goals under constraints; means–ends, sequencing, trade-offs.
- **Argumentative reasoning:** construct, evaluate, and rebut arguments; justify conclusions to self/others.
- **Metacognitive reasoning:** monitor and regulate one's own thinking (confidence, error checking, deciding when to stop/search more).

IS HUMAN REASONING PERFECT ?

- **Bounded rationality**: humans don't optimize globally; we reason under limited time, attention, and information, so we satisfice and use shortcuts.
- **Heuristics can be efficient but systematically biased**: fast rules of thumb work often, but create predictable errors (base-rate neglect, availability, anchoring).
- **Mental models are capacity-limited**: people reason by simulating a small set of possible situations; errors arise when critical alternatives aren't represented.
- **Deduction in natural language is not formal logic**: interpretation and content/framing strongly affect performance; people often seek confirming evidence.
- **Planning/search is constrained**: humans use heuristic search, subgoals, and stopping rules rather than exhaustive exploration.
- **Reasoning is often argumentative**: people are better at generating and defending explanations than neutrally checking them, especially when motivated
- **Seminal works:**
  - Simon, H. A. (1955). A behavioral model of rational choice. *Quarterly Journal of Economics, 69*(1), 99–118.
  - Johnson-Laird, P. N. (1983). Mental models: Towards a cognitive science of language, inference, and consciousness. Cambridge, MA: Harvard University Press.
  - Tversky, A., & Kahneman, D. (1974). Judgment under uncertainty: Heuristics and biases. *Science, 185*(4157), 1124–1131.
  - Newell, A., & Simon, H. A. (1972). *Human problem solving.* Englewood Cliffs, NJ: Prentice-Hall.
- **Spoiler:** Not even close :) !

SO WHAT DO LLMS EMULATE (OR APPROXIMATE) ?

- **Analogical reasoning** (often strongest match)
  - Pure LLMs are excellent at mapping a new prompt onto a familiar structural template ("this looks like X"), because pretraining is essentially learning a gigantic space of near-paraphrases and pattern families.
  - Failure mode: surface analogy beats deep invariants; the model "fits the vibe" and misses constraints when the structure is subtle.
- **Abductive reasoning / hypothesis generation** (very strong, but not grounded)
  - LLMs readily propose plausible explanations (diagnoses, causes, interpretations) because abduction is basically "most probable story given context."
  - Failure mode: plausibility can outrun truth; fluent explanations can be overconfident and internally coherent but wrong.
- **Inductive reasoning / statistical generalization** (strong within-distribution)
  - Pretraining gives powerful inductive generalization over text distributions; models interpolate well in familiar regimes.
  - Failure mode: distribution shift and "small perturbations" can cause disproportionate changes in output quality: robustness is not guaranteed.
- **Deductive reasoning (partial, brittle)**
  - LLMs can execute short chains of rule-like entailment when the pattern is common and variable binding is easy.
  - Failure modes: fragile variable tracking / bookkeeping; sensitivity to irrelevant-but-plausible clauses (spurious operations); long-horizon deductions collapse into confident rationalizations.

SO WHAT DO LLMS EMULATE (OR APPROXIMATE) ?

- **Procedural / algorithmic reasoning** (moderate, horizon-limited)
  - LLMs often imitate known procedures (algebra steps, coding idioms) and benefit from "writing out" intermediate steps because the extra tokens create scaffolding/context for the next prediction.
  - Failure mode: the model is not running a guaranteed algorithm; it can drift, skip constraints, or "complete" a familiar-looking procedure incorrectly.
- **Planning / practical reasoning** (high plausibility, low validity)
  - Pure LLMs generate coherent plans and subgoals (means–ends narrative).
  - Failure mode: state tracking is weak: plans sound locally sensible but can violate preconditions, forget earlier commitments, or fail globally.
- **Causal / counterfactual reasoning** (strong storytelling, weak counterfactual discipline)
  - LLMs produce causal narratives and mechanism-like explanations.
  - Failure mode: counterfactuals require stable causal models; pure LLMs often substitute narrative coherence for intervention logic.
- **Metacognitive reasoning / self-verification** (weakest in "pure" form)
  - Self-critique is typically "another sampled completion," not an independent checking process. Without an external criterion, it often fails to reliably detect its own mistakes (and can introduce new ones).

LET'S EXAMINE

- A) Mirzadeh et al. (2025). *GSM-Symbolic: Understanding the Limitations of Mathematical Reasoning in Large Language Models.* ICLR 2025.
- B) Valmeekam et al. (2025). *On the Self-Verification Limitations of Large Language Models on Reasoning and Planning Tasks.* ICLR 2025.
  - Kambhampati, S., Stechly, K., & Valmeekam, K. (2025). *(How) Do reasoning models reason?*. Annals of the New York Academy of Sciences, 1547(1), 33-40

REASONING, IS STILL BRITTLE

- We're not debating whether LLMs can solve hard problems sometimes; we're isolating **where reliability collapses** even when they look strong on headline benchmarks.
  - Paper A (**GSM-Symbolic**) attacks the **evaluation illusion**: one benchmark score hides instability; when you generate many equivalent versions, accuracy becomes a **distribution** with large spread.
  - Paper B (**Self-verification limitations**) attacks the **prompting illusion**: "just ask it to check itself" often fails; critique loops can *decrease* correctness, especially in reasoning/planning.
- Unifying lens: "reasoning" is not one skill; it's a pipeline:
  - parse text → formal structure
  - choose relevant facts/operations
  - maintain constraints/state through steps
  - verify correctness

BENCHMARKS GALORE

- MMLU : Broad multi-subject multiple-choice exam covering humanities, STEM, social sciences; used as a general knowledge/reasoning snapshot. https://huggingface.co/datasets/cais/mmlu
- MMLU-Pro : A harder/cleaner variant of MMLU-style evaluation with increased difficulty and improved filtering; used to reduce "easy win" effects. https://github.com/TIGER-AI-Lab/MMLU-Pro
- GPQA : Graduate-level, "hard to bluff" science Q&A intended to stress genuine expert reasoning rather than surface familiarity. https://github.com/idavidrein/gpqa
- GSM8K : Grade-school math word problems; stresses multi-step quantitative reasoning (often evaluated with chain-of-thought prompting). https://github.com/openai/grade-school-math
- HumanEval : Code generation problems evaluated by unit tests; measures functional correctness of synthesized code. https://github.com/openai/human-eval
- MBPP : Mostly basic Python programming problems (often shorter than HumanEval); also used for code synthesis evaluation. https://github.com/google-research/google-research/tree/master/mbpp
- SWE-bench : Real GitHub issues mapped to codebase patches; evaluates end-to-end software engineering (edit, fix, avoid regressions). https://github.com/swe-bench/SWE-bench
- MedQA : USMLE-style multiple-choice medical exam questions; tests medical knowledge + clinical reasoning in exam format. https://github.com/jind11/MedQA
- MedMCQA : Large-scale medical multiple-choice QA (typically from medical entrance/education sources); used for medical knowledge testing. https://github.com/medmcqa/medmcqa
- PubMedQA : Biomedical question answering grounded in PubMed abstracts; often used to test evidence-based biomedical QA. https://github.com/pubmedqa/pubmedqa

EXAMINING MODEL REASONING CAPABILITIES: TWO SCHOOLS

- Asses the intelligence externally
- Open it up

GSM8K-STYLE ACCURACY IS A WEAK SCIENTIFIC CLAIM

- A single accuracy number on a static test set is:
  - a **point estimate** (no notion of variance)
  - vulnerable to **dataset peculiarities** (style, common numbers, recurring patterns)
  - potentially inflated by **training exposure** (direct memorization isn't required; partial/template exposure is enough)
- Even if two models score 85% on GSM8K, you don't know:
  - which problem *families* they truly handle
  - whether success is stable under small changes
  - whether failures are rare noise or systematic brittleness
- GSM-Symbolic replaces "one test set" with "many controlled variants," so robustness becomes measurable.
- GSM8K: https://github.com/openai/grade-school-math

KEY CONSTRUCTION: FROM A WORD PROBLEM TO A SYMBOLIC TEMPLATE

- Take a GSM8K problem and rewrite it as:
  - a **template** with named variables (numbers, entities)
  - **constraints** to keep instances well-formed (e.g., integer answers, positivity, plausible quantities)
  - a generator that samples new values satisfying constraints
- So we can now generate:
  - 1, 10, 50… entire *alternate test sets* with the same underlying reasoning structure
- Conceptual shift: **the unit of evaluation is the template**, not the single instance.
- This is closer to how we evaluate algorithms: do they solve a class, not a memorized item.

SYMBOLIC TEMPLATE

GSM8K

When Sophie watches her nephew, she gets out a variety of toys for him. The bag of building blocks has 31 blocks in it. The bin of stuffed animals has 8 stuffed animals inside. The tower of stacking rings has 9 multicolored rings on it.Sophie recently bought a tube of bouncy balls, bringing her total number of toys for her nephew up to 62. How many bouncy balls came in the tube?

Let T be the number of bouncy balls in the tube.
After buying the tube of balls, Sophie has 31+8+9+ T = 48 + T =62 toys for her nephew.
Thus, T =62-48 = <<62-48=14>>14 bouncy balls came in the tube.

GSM Symbolic Template

When {name} watches her {family}, she gets out a variety of toys for him. The bag of building blocks has {x} blocks in it. The bin of stuffed animals has {y} stuffed animals inside.The tower of stacking rings has {z} multicolored rings on it.{name} recently bought a tube of bouncy balls, bringing her total number of toys she bought for her {family} up to {total}. How many bouncy balls came in the tube?

#variables:
- name = sample(names)
- family = sample(["nephew", "cousin", "brother"])
- x = range(5, 100)
- y = range(5, 100)
- z = range(5, 100)
- total = range(100, 500)
- ans = range(85, 200)

#conditions:
- x + y + z + ans == total

Let T be the number of bouncy balls in the tube. After buying the tube of balls, {name} has {x} + {y} + {z} + T = { x + y + z } + T = {total} toys for her {family}.

Thus, T = {total} - { x + y + z } = <<{total}-{ x + y + z }={ans}>>{ans} bouncy balls came in the tube.

EXISTING BENCHMARKS: CAVEATS

- Instead of "accuracy," we get:
  - **mean accuracy** across instantiations
  - **variance** across instantiations
  - **tail risk**: how bad does it get on unlucky instantiations?
- We can ask targeted questions:
  - Are models invariant to renaming entities?
  - Are they stable under numeric perturbations?
  - Do they degrade gracefully with added conditions?
  - Can they ignore irrelevant-but-plausible statements?
- This is the right shape of evaluation for "reasoning systems," not just "text generators."

THE INVARIANCE PRINCIPLE: WHAT SHOULD *NOT* CHANGE IF YOU CAN REASON

- If a student truly understands a problem type, these should mostly not matter:
  - swapping "Alice/Bob" with "Maya/Chen"
  - changing 12 apples to 17 apples (same structure)
  - adding irrelevant narrative detail that does not affect the computation
- The paper's tests are essentially **invariance tests**:
  - the "mathematical program" is the same
  - only the surface form changes
- If correctness swings wildly, the model is not executing a stable internal program; it's likely relying on brittle cues.

EXPERIMENT AXIS 1: NAMES-ONLY VS NUMBERS-ONLY PERTURBATIONS

- Names-only change:
  - typically just placeholder substitution
  - should have near-zero effect if the model is structure-driven
- Numbers-only change:
  - preserves the *logic of steps* but changes the arithmetic trajectory
  - a robust solver should handle it consistently
- Reported pattern (across many models):
  - name perturbations: smaller drop/variance
  - number perturbations: **larger drop and noticeably larger variance**
- Interpretation: numbers disrupt "familiar-looking" intermediate states; models appear to use stereotyped trajectories more than stable procedure.

WHY NUMBER PERTURBATION IS A DEEPER TEST THAN IT SOUNDS

- This is not just "can you add correctly?"
- Number changes stress:
  - divisibility and "clean" intermediate values (some numbers lead to messier steps)
  - whether the model is doing real constraint tracking vs guessing
  - whether it can recompute rather than recall a known pattern
- In many LLM failures, arithmetic is fine *conditional on the wrong setup*. Numbers change tends to expose **setup brittleness**.

EXPERIMENT AXIS 2: COMPOSITIONAL DEPTH VIA CLAUSE ADDITIONS

- They systematically modify problems by adding extra clauses/conditions (think: another discount, another step, another constraint).
- Expected behavior if reasoning is algorithmic:
  - accuracy might drop gradually (more steps, more chances to slip)
  - but performance should be relatively stable across instantiations of the same template depth
- Reported behavior:
  - mean accuracy drops
  - variance increases with depth (wider spread across instantiations)
- That widening is the key: it suggests instability in the internal reasoning trajectory, not just occasional slips.

VARIANCE GROWS WITH DEPTH: THIS IS A PROBLEM !

- In applied settings, deeper reasoning is common (multi-symptom triage, multi-policy compliance, multi-hop RCA).
- If variance expands with depth, then:
  - a system can look great on curated examples
  - but behave unreliably in the wild where values and contexts vary
- Depth increases:
  - opportunities for mis-parsing
  - opportunities for wrong operation selection
  - sensitivity to earlier small errors (compounding)
- This foreshadows why “long chain-of-thought” is not automatically safer.

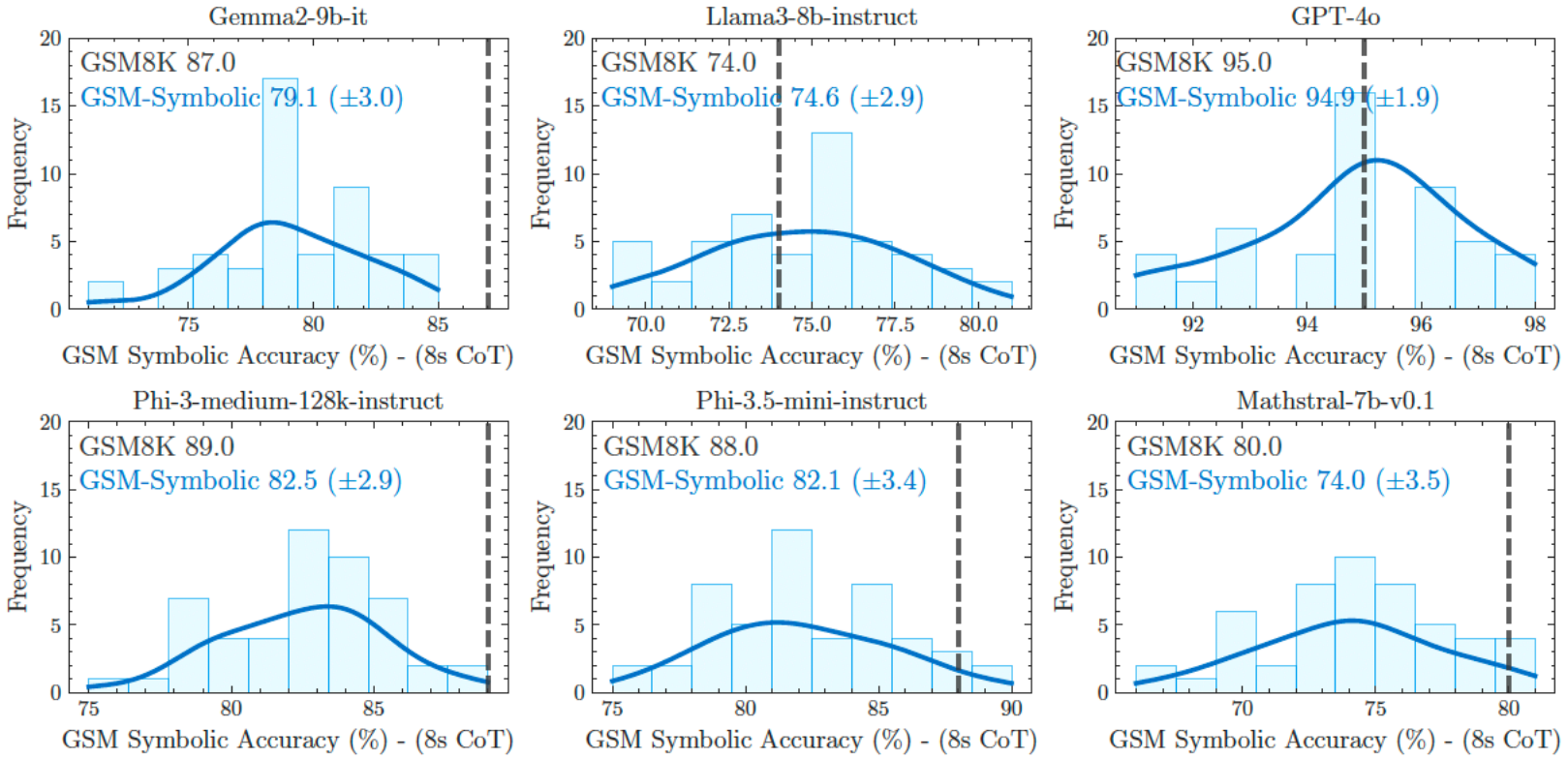
GSM-NOOP: THE "IRRELEVANT CLAUSE" TRAP

- Their most diagnostic variant: add a clause that *sounds actionable* but should not change the computation.
- This is a test of **relevance discrimination**:
  - Can the model decide which facts are operationally relevant?
  - Or does it automatically map "numbers + comparative language" to an operation?
- Classic failure pattern:
  - "five were smaller than average" → model subtracts 5
    - because "smaller" frequently co-occurs with subtraction patterns in training
- This is not a math error; it's a semantic compilation error: language → operation.

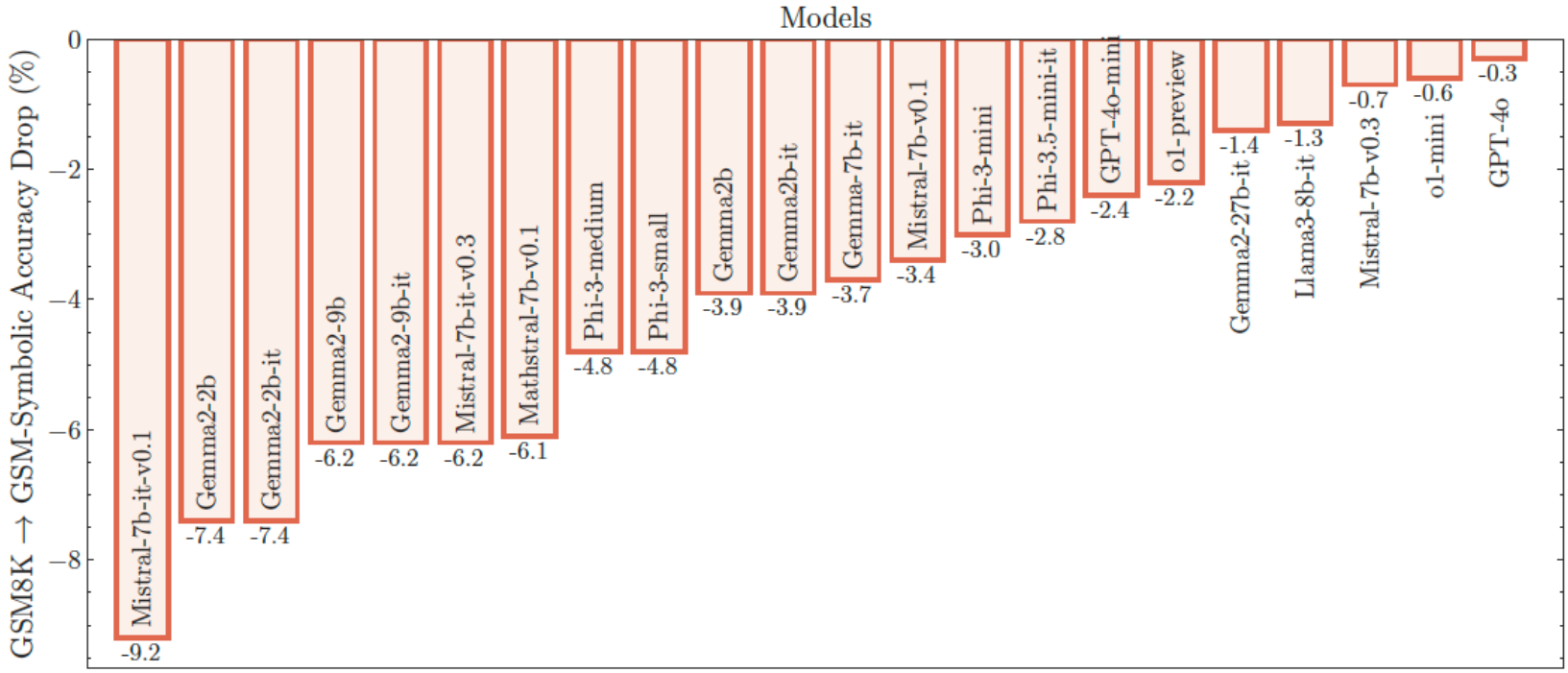
NOOP IS A GENUINE REASONING TEST

- Real reasoning requires:
  - mapping natural language to a *formal* structure
  - ignoring irrelevant detail (humans do this effortlessly)
- NoOp measures whether the model:
  - truly grounds the clause in the formal structure
  - or just pattern-matches words to operations
- A model can produce beautiful step-by-step text while:
  - applying an unwarranted operation
  - then correctly finishing the wrong program
- That's exactly why chain-of-thought can be misleading as "evidence of reasoning."

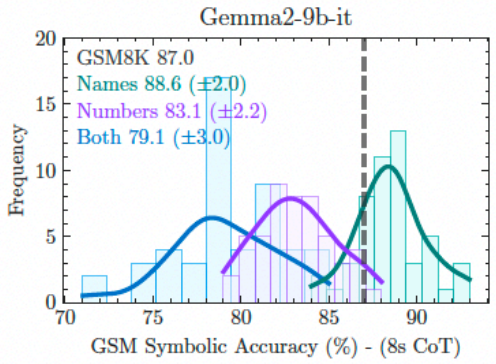
RESULTS

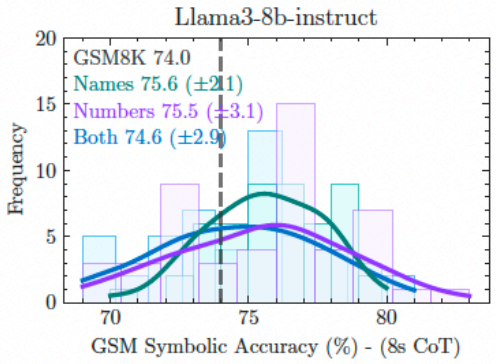

Gemma2-9b-it
GSM8K 87.0
GSM-Symbolic 79.1 (±3.0)
Frequency
GSM Symbolic Accuracy (%) - (8s CoT)
Llama3-8b-instruct
GSM8K 74.0
GSM-Symbolic 74.6 (±2.9)
Frequency
GSM Symbolic Accuracy (%) - (8s CoT)
GPT-4o
GSM8K 95.0
GSM-Symbolic 94.9 (±1.9)
Frequency
GSM Symbolic Accuracy (%) - (8s CoT)
Phi-3-medium-128k-instruct
GSM8K 89.0
GSM-Symbolic 82.5 (±2.9)
Frequency
GSM Symbolic Accuracy (%) - (8s CoT)
Phi-3.5-mini-instruct
GSM8K 88.0
GSM-Symbolic 82.1 (±3.4)
Frequency
GSM Symbolic Accuracy (%) - (8s CoT)
Mathstral-7b-v0.1
GSM8K 80.0
GSM-Symbolic 74.0 (±3.5)
Frequency
GSM Symbolic Accuracy (%) - (8s CoT)

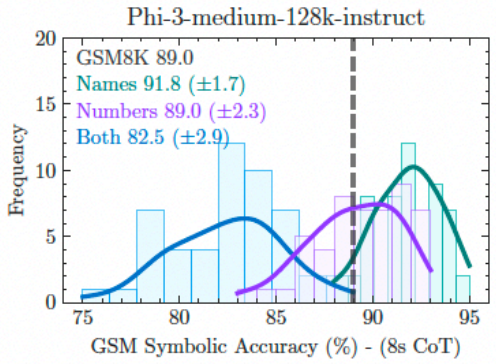
RESULTS

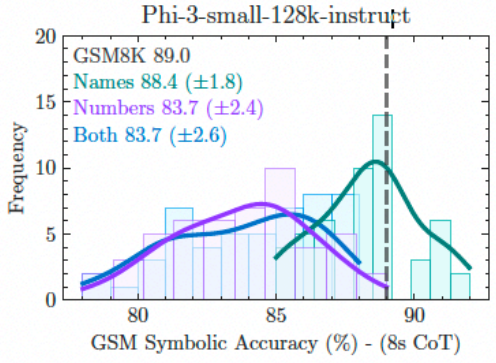

Models
GSM8K → GSM-Symbolic Accuracy Drop (%)
0
−2
−4
−6
−8
Mistral-7b-it-v0.1
-9.2
Gemma2-2b
-7.4
Gemma2-2b-it
-7.4
Gemma2-9b
-6.2
Gemma2-9b-it
-6.2
Mistral-7b-it-v0.3
-6.2
Mathstral-7b-v0.1
-6.1
Phi-3-medium
-4.8
Phi-3-small
-4.8
Gemma2b
-3.9
Gemma2b-it
-3.9
Gemma-7b-it
-3.7
Mistral-7b-v0.1
-3.4
Phi-3-mini
-3.0
Phi-3.5-mini-it
-2.8
GPT-4o-mini
-2.4
o1-preview
-2.2
Gemma2-27b-it
-1.4
Llama3-8b-it
-1.3
Mistral-7b-v0.3
-0.7
o1-mini
-0.6
GPT-4o
-0.3

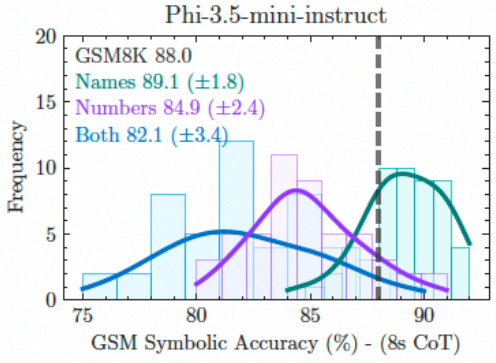
RESULTS

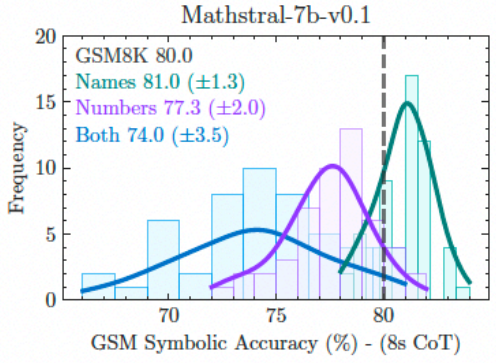

Gemma2-9b-it
GSM8K 87.0
Names 88.6 (±2.0)
Numbers 83.1 (±2.2)
Both 79.1 (±3.0)
Frequency
GSM Symbolic Accuracy (%) - (8s CoT)


Llama3-8b-instruct
GSM8K 74.0
Names 75.6 (±2.1)
Numbers 75.5 (±3.1)
Both 74.6 (±2.9)
Frequency
GSM Symbolic Accuracy (%) - (8s CoT)


Phi-3-medium-128k-instruct
GSM8K 89.0
Names 91.8 (±1.7)
Numbers 89.0 (±2.3)
Both 82.5 (±2.9)
Frequency
GSM Symbolic Accuracy (%) - (8s CoT)


Phi-3-small-128k-instruct
GSM8K 89.0
Names 88.4 (±1.8)
Numbers 83.7 (±2.4)
Both 83.7 (±2.6)
Frequency
GSM Symbolic Accuracy (%) - (8s CoT)


Phi-3.5-mini-instruct
GSM8K 88.0
Names 89.1 (±1.8)
Numbers 84.9 (±2.4)
Both 82.1 (±3.4)
Frequency
GSM Symbolic Accuracy (%) - (8s CoT)


Mathstral-7b-v0.1
GSM8K 80.0
Names 81.0 (±1.3)
Numbers 77.3 (±2.0)
Both 74.0 (±3.5)
Frequency
GSM Symbolic Accuracy (%) - (8s CoT)

RESULTS

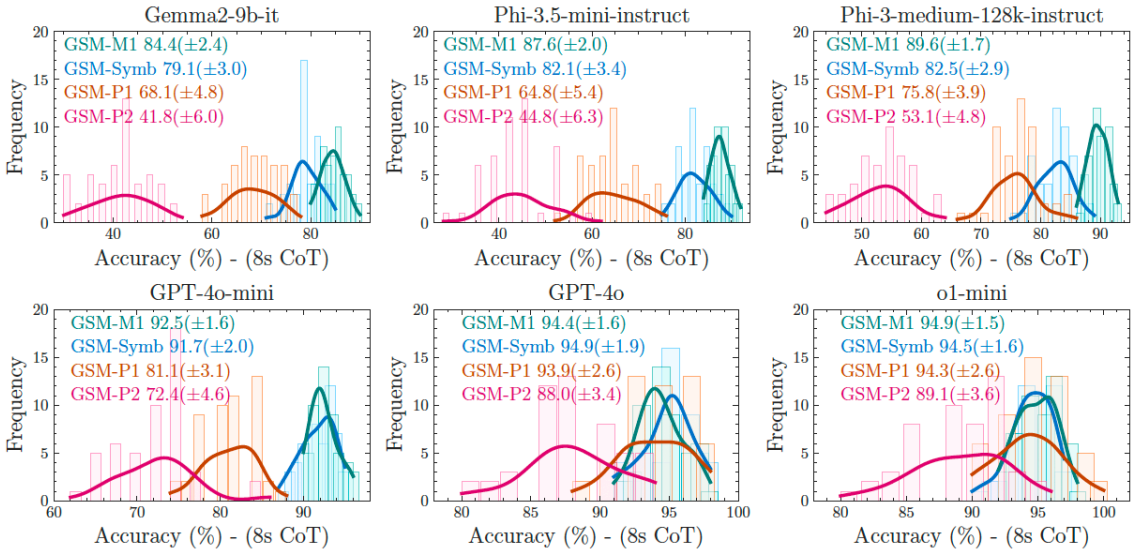
Different Levels of GSM-Symbolic Difficulty

GSM-Symbolic-M1: To make a call from a phone booth, you must pay $0.6 for each minute of your call. ~~After 10 minutes, that price drops to $0.5 per minute~~.How much would a 60-minute call cost?

GSM-Symbolic: To make a call from a phone booth, you must pay $0.6 for each minute of your call. After 10 minutes, that price drops to $0.5 per minute. How much would a 60-minute call cost?

GSM-Symbolic-P1: To make a call from a hotel room phone, you must pay $0.6 for each minute of your call.After 10 minutes, that price drops to $0.5 per minute. After 25 minutes from the start of the call, the price drops even more to $0.3 per minute.How much would a 60-minute call cost?

GSM-Symbolic-P2: To make a call from a hotel room phone, you must pay $0.6 for each minute of your call. After 10 minutes, the price drops to $0.5 per minute. After 25 minutes from the start of the call, the price drops even more to $0.3 per minute. If your total bill is more than $10, you get a 25% discount. How much would a 60-minute call cost?



NOOP

GSM-NoOp

Oliver picks 44 kiwis on Friday. Then he picks 58 kiwis on Saturday. On Sunday, he picks double the number of kiwis he did on Friday, but five of them were a bit smaller than average. How many kiwis does Oliver have?

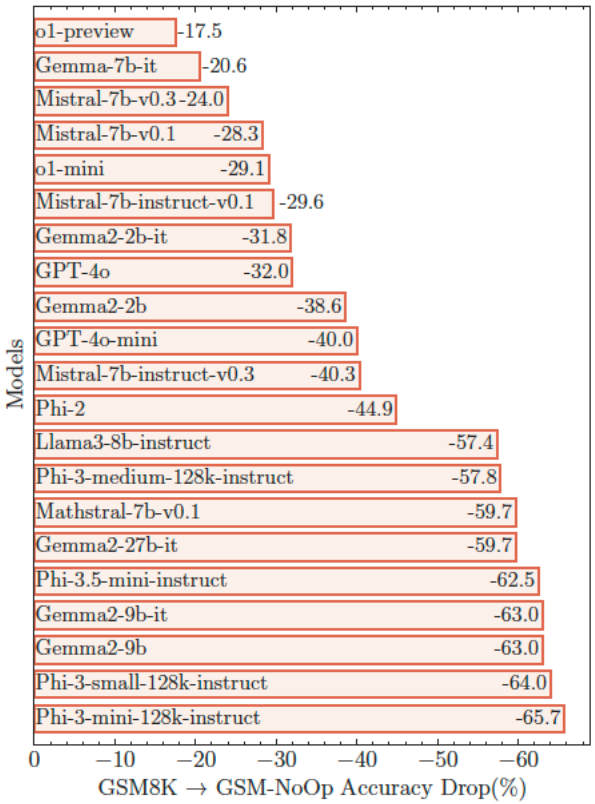
---

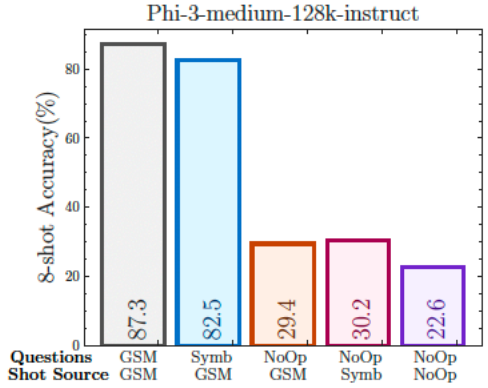
o1-mini: Oliver picks kiwis over three days as follows:
- Friday: 44 kiwis
- Saturday: 58 kiwis
- Sunday: Double the number he picked on Friday, which is $2 \times 44 = 88$ kiwis

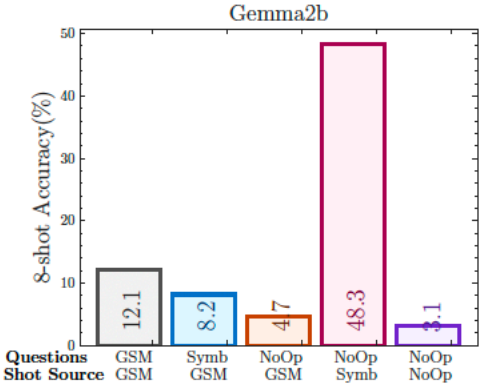
However, on Sunday, 5 of these kiwis were smaller than average. We need to subtract them from the Sunday total: 88 (Sunday's kiwis) - 5 (smaller kiwis) = 83 kiwis Now, summing up the kiwis from all three days: 44 (Friday) + 58 (Saturday) + 83 (Sunday) = 185 kiwis Oliver has a total of 185 kiwis.

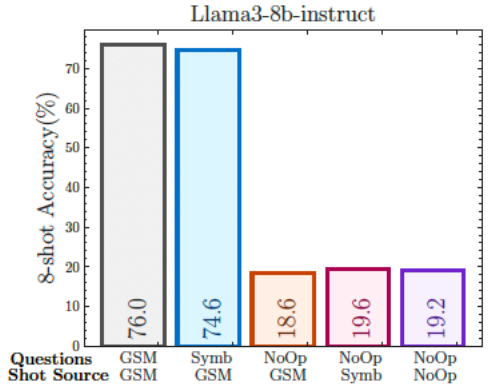
Llama3-8B: Let's break it down step by step:
Oliver picks 44 kiwis on Friday. He picks 58 kiwis on Saturday. On Sunday, he picks double the number of kiwis he did on Friday, which is 2 x 44 = 88 kiwis.
Five of the kiwis picked on Sunday are a bit smaller than average, so we subtract 5 from the total number of kiwis picked on Sunday: 88 - 5 = 83 kiwis. Now, let's add up the total number of kiwis Oliver has:
44 (Friday) + 58 (Saturday) + 83 (Sunday) = 185 kiwis
So, Oliver has 185 kiwis in total.

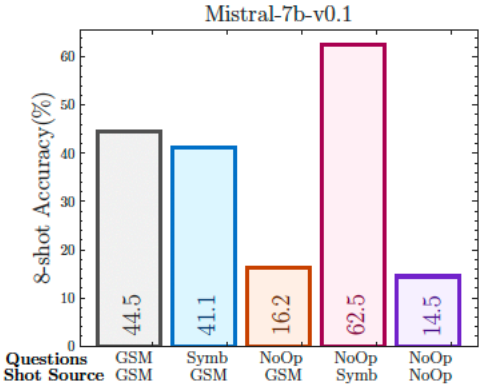
RESULTS


Models
o1-preview -17.5
Gemma-7b-it -20.6
Mistral-7b-v0.3 -24.0
Mistral-7b-v0.1 -28.3
o1-mini -29.1
Mistral-7b-instruct-v0.1 -29.6
Gemma2-2b-it -31.8
GPT-4o -32.0
Gemma2-2b -38.6
GPT-4o-mini -40.0
Mistral-7b-instruct-v0.3 -40.3
Phi-2 -44.9
Llama3-8b-instruct -57.4
Phi-3-medium-128k-instruct -57.8
Mathstral-7b-v0.1 -59.7
Gemma2-27b-it -59.7
Phi-3.5-mini-instruct -62.5
Gemma2-9b-it -63.0
Gemma2-9b -63.0
Phi-3-small-128k-instruct -64.0
Phi-3-mini-128k-instruct -65.7
0 −10 −20 −30 −40 −50 −60
GSM8K → GSM-NoOp Accuracy Drop(%)


Phi-3-medium-128k-instruct
8-shot Accuracy(%)
87.3
82.5
29.4
30.2
22.6
Questions GSM Symb NoOp NoOp NoOp
Shot Source GSM GSM GSM Symb NoOp


Gemma2b
8-shot Accuracy(%)
12.1
8.2
4.7
48.3
3.1
Questions GSM Symb NoOp NoOp NoOp
Shot Source GSM GSM GSM Symb NoOp


Llama3-8b-instruct
8-shot Accuracy(%)
76.0
74.6
18.6
19.6
19.2
Questions GSM Symb NoOp NoOp NoOp
Shot Source GSM GSM GSM Symb NoOp


Mistral-7b-v0.1
8-shot Accuracy(%)
44.5
41.1
16.2
62.5
14.5
Questions GSM Symb NoOp NoOp NoOp
Shot Source GSM GSM GSM Symb NoOp

FEW-SHOT DOESN'T RELIABLY FIX NOOP

- If the issue were "didn't know how," then showing multiple similar demonstrations should strongly help.
- But, even with multiple examples, NoOp failures persist
  - The improvements are not robust
- Implication: the failure is **habitual** (a learned mapping from cues → ops), not a missing pattern.
- This matters for course projects: "add more exemplars" is not a general safety strategy.

BASICALLY

- Replace “LLM reasoning skill = benchmark score” with:
  - “reasoning skill = stability under semantics-preserving perturbations”
- Two deep signals the paper reveals:
  - **distributional instability** (variance across equivalent instances)
  - **relevance failures** (NoOp causes spurious operations)
- Engineering translation:
  - robustness tests should be part of any evaluation for reasoning-heavy systems
  - and systems should separate “interpretation of text” from “execution of computation” wherever possible

SELF-VERIFICATION

THE POPULAR BELIEF: "CHECKING IS EASIER THAN SOLVING"

- Many prompting recipes assume:
  - generate an answer
  - ask the model to critique it
  - revise using critique
  - repeat → converge to correct
- The intuition is computational ("verification is simpler than synthesis").
- The paper asks: empirically, for LLMs, is that actually true on reasoning and planning tasks with crisp correctness?
- The key twist: LLM "verification" is itself another generation step, not an interpreter.

TASKS WITH HARD VALIDITY CRITERIA

- Choose tasks where correctness can be unambiguous (not "sounds plausible"):
  - arithmetic/constraint-style reasoning
  - combinatorial validity constraints
  - planning tasks where actions have preconditions/effects and goals must be satisfied
- This avoids the common failure of critique research: grading the critique by style or plausibility rather than correctness.

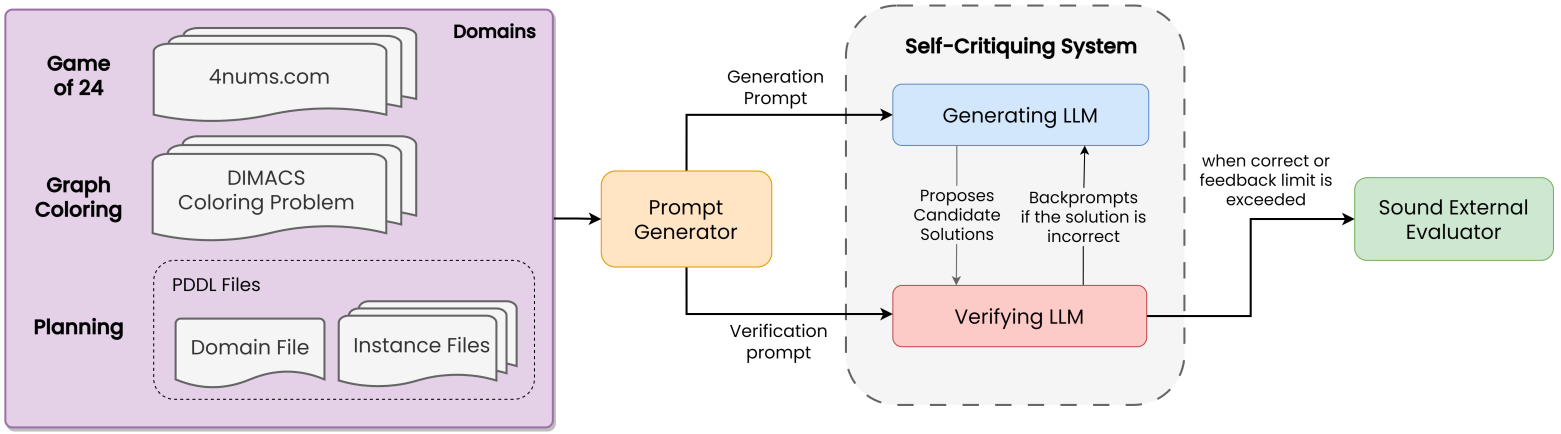
DECOMPOSE SELF-VERIFICATION INTO THREE DISTINCT ABILITIES

- The loop actually requires *three* skills:
  - **Verifier skill:** detect whether a solution is correct/incorrect
  - **Critique skill:** identify what is wrong and why
  - **Revision skill:** use critique to produce a corrected solution
- Most "self-critique" demonstrations treat it as one monolithic behavior.
- This proposal isolates them because failure in any one breaks the loop.

SELF-CRITIQUE OFTEN HURTS !

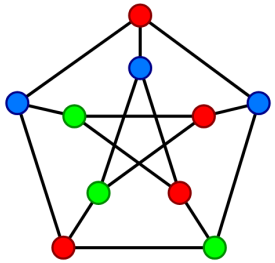
- On many reasoning/planning tasks:
  - self-critique does not reliably improve accuracy
  - and on harder settings it can **collapse** performance
- Why collapse is plausible:
  - critique can introduce new errors
  - the revision may overfit to a wrong critique
  - the model may become more confident in a wrong path
- This is a sober correction to “reflection always helps.”

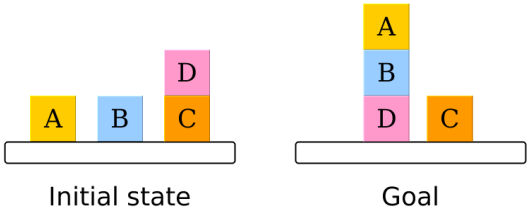
EVALUATION PIPELINE



- 3 Problems
  - Game of 24
  - Graph coloring
  - Planning (Blocksworld)

(10 - (8 / 2)) X 6 ?





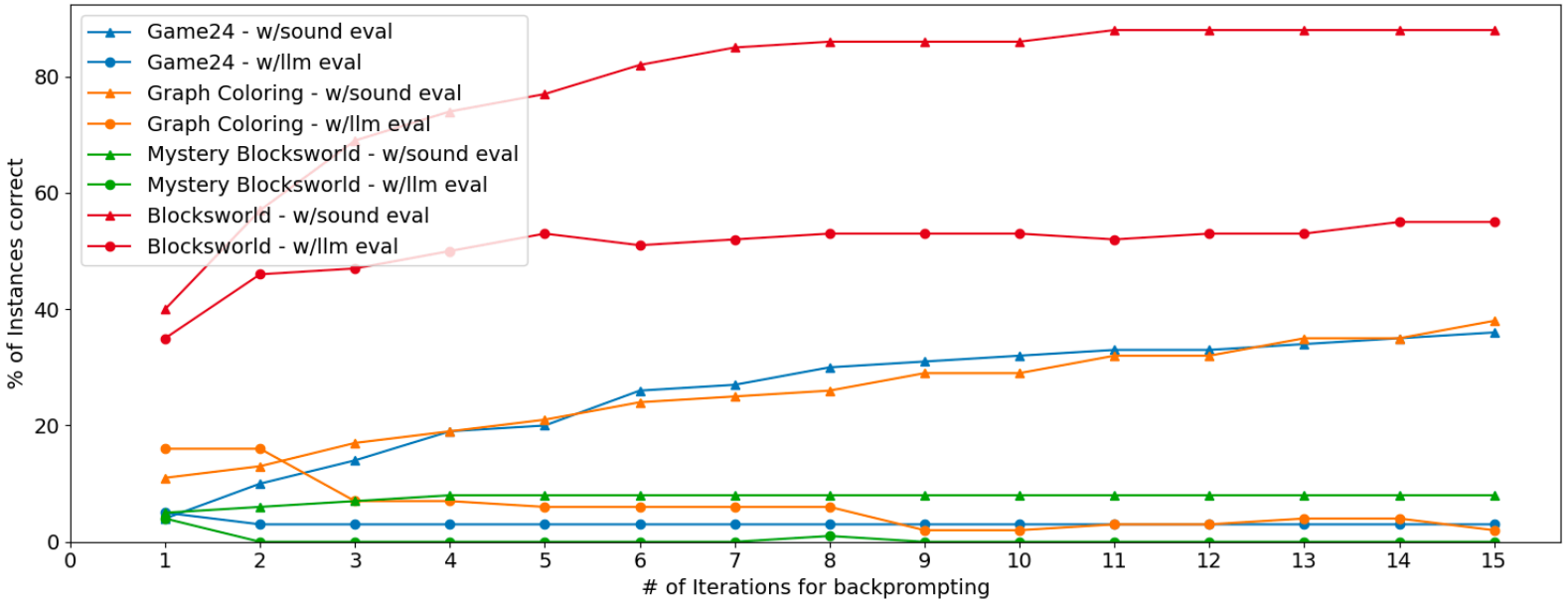
EVALUATION METHODOLOGY

- **Core object of study:** an *iterative backprompting loop* that people informally call "self-critique," broken into separable roles:
  - 1. **Guesser** (proposes a solution)
  - 2. **Verifier** (binary "correct?")
  - 3. **Critique generator** (free-form feedback describing what's wrong)
- Loop architecture (same across domains):
  - Start with a **formal instance** (Game of 24 numbers; graph + chromatic number; PDDL planning instance).
  - A **hard-coded translator** turns the instance into a natural-language prompt.
  - LLM produces an **answer candidate** (expression / coloring / plan).
  - That candidate is wrapped into a **verification + critique prompt** and sent to a verifier (either the LLM itself or a sound external checker).
  - If verifier says "correct," stop and return the candidate.
  - If "incorrect," extract critique and append it to a running **history prompt** that includes *all previous guesses + feedback*, then query again.
  - Repeat until **timeout** (implemented as max iterations).
- **Prompt includes full history:** intentionally give the self-correction loop the *best possible chance* by exposing all prior failures and feedback, so if "self-critique helps," it should show up here.

EVALUATION

- Domains and "sound verifiers" used (critical design choice):
  - **Game of 24:** *symbolic evaluation checker* (detect wrong value, malformed parentheses, wrong numbers used).
  - **Graph coloring:** *programmatic edge-checker* (for each edge, verify endpoints have different colors).
  - **STRIPS planning (Blocksworld + Mystery Blocksworld):** *plan validator* that checks action preconditions/effects and goal reachability.
- What is measured (two layers):
  - **End-to-end accuracy** of the whole loop under different prompting schemes (Table 1).
  - **Verifier quality** of the LLM-as-verifier: accuracy, false positives, false negatives (Table 2). (this is key because if the verifier is unreliable, the loop can get strictly worse with more iterations.)
- **Ablations are the whole point:** systematically replace/remove pieces to answer: *"Are improvements coming from critique content, or just from having multiple guesses plus a correct accept/reject signal?"*
- Three feedback richness regimes (when using sound verifier):
  - **Binary Feedback (BF):** "wrong" (no details).
  - **First Error Feedback (FEF):** point out the first detected violation (first wrong edge / first invalid plan step / expression evaluation not 24).
  - **All Error Feedback (AEF):** list all detected violations (all wrong edges / all invalid steps).
    - This tests whether "more critique information" actually guides the generator.
- Sampling baseline (crucial ablation):
  - Keep the **sound verifier**, but **remove critique entirely**: re-ask the *same base prompt* repeatedly, just sampling multiple independent guesses until one passes or timeouts.
  - This isolates whether critique text is causally useful, versus "more chances + a correct checker."

ACCURACY VS ITERATIONS



ACCURACY VS ITERATIONS

- **x-axis:** number of allowed iterations/backprompts before timeout (if we give the system more turns to 'self-correct,' does it get better?)
- **y-axis:** percent of instances solved by iteration n,
  - counted as: instances that are actually correct by iteration n (whether or not the LLM verifier recognized them), **plus** those that have been accepted by the verifier so far.
- Key observation: monotonic improvement requires a sound verifier !
  - With a **sound verifier**, performance rises with more iterations and approaches an asymptote.
    - Why monotonic?
- Why it can *decrease* between iterations:
  - If the verifier incorrectly **rejects** a correct answer (false negative), the system continues.
- Basically:
  - When the **LLM is used as its own verifier**, performance collapses quickly with more iterations.
  - Interpretable reason: the LLM-verifier fails to reliably detect correct answers; the loop then spirals into later, lower-quality guesses and hits timeout.
- **Iterating only helps when there is a reliable accept/reject criterion !**

RESULTS

Domain	S.P.	LLM+LLM	LLM+Sound Critique			Sampling		S.C.
			B.F.	F.E.F	A.E.F	k=15	k=25	k=15
Game of 24	5%	3%	36%	38%	N/A	28%	42%	6%
Graph Coloring	16%	2%	38%	37%	34%	40%	44%	14%
Blocksworld	40%	55%	60%	87%	83%	68%	72%	42%
Mystery Blocksworld	4%	0%	10%	8%	6%	9%	14%	4%

- Self-critique with GPT-4 as its own verifier can be dramatically worse than one-shot.
  - Graph Coloring drops from 16% (standard prompting) to 2% (LLM+LLM self-critique); Mystery Blocksworld drops from 4% to 0%.
- Replacing the verifier with a sound verifier dominates: huge jumps across all domains.
  - Game of 24: 5% → 36–38% (Binary / First-Error feedback); Graph Coloring: 16% → 34–38% (All-Error / Binary feedback); Blocksworld: 40% → 83–87% (All-Error / First-Error feedback); Mystery Blocksworld: 4% → 6–10% (All-Error / Binary feedback).
- Critique detail is not the main lever; sometimes "more detailed" is not better.
  - Blocksworld: First-Error 87% vs All-Error 83% (more feedback didn't help).
  - Graph Coloring: Binary 38% vs All-Error 34%.
- Ablating critique entirely (just resampling + sound verifier) keeps most of the gains.
  - Graph Coloring: Sampling-25 = 44% (beats Binary 38%, All-Error 34%).
  - Game of 24: Sampling-25 = 42% (beats First-Error 38%).
  - Blocksworld: Sampling-25 = 72% (lower than 87% but still far above 40%).

RESULTS

Domain	Accuracy	F.P.R	F.N.R
Game of 24	87.0% (3567/4100)	10.4% (320/3071)	20.7% (213/1029)
Graph Coloring	72.4% (362/500)	6.5% (25/382)	95.8% (113/118)
Blocksworld	71.8% (359/500)	18.55% (64/345)	15.48% (24/155)
Mystery Blocksworld	79.6% (398/500)	0.5% (2/397)	97.09% (100/103)

- Table 2 explains the **self-critique collapses:** GPT-4-as-verifier has extreme false negatives in the hard domains.
  - Graph Coloring verifier: FNR 95.8% (rejects 113/118 correct solutions); accuracy 72.4%.
  - Mystery Blocksworld verifier: FNR 97.09% (rejects 100/103 correct solutions); accuracy 79.6%.
  - This means the loop often throws away correct answers and keeps iterating into worse ones/timeouts.
- Where verifier false negatives are lower, LLM+LLM doesn't crater as badly (and can even improve).
  - Blocksworld: baseline 40%, LLM+LLM 55%; verifier FNR is only 15.48%.
  - Game of 24: baseline 5%, LLM+LLM 3%; verifier FNR 20.7% (still not great).

WHAT ACTUALLY HELPS: EXTERNAL, SOUND VERIFICATION + SELECTION

- The paper's practical lesson: gains come from **sound external checks**.
- Once we have a verifier, we can:
  - sample multiple candidates from the LLM
  - run the verifier
  - select a valid candidate

WHY “MANY TRIES” WORKS BETTER THAN “ONE PERFECT CRITIQUE”

- Search over candidates is powerful when:
  - the verifier is correct
  - the space contains at least one valid solution with reasonable probability
- Critique content may be much less important than:
  - the ability to generate diverse candidates
  - the presence of an acceptance test
- This is the same idea as randomized algorithms + a checker: correctness comes from the checker.

SELF-CONSISTENCY IS NOT VERIFICATION

- Self-consistency (majority vote across multiple chains) relies on an assumption:
  - errors are not highly correlated
- But in LLMs, errors are often correlated because:
  - the same misleading cues trigger the same wrong operation
  - the same hidden misconception drives multiple generations
- Majority vote can amplify confident wrongness.
- Verification requires an external criterion (execution, constraints, solver), not internal consensus.

COMMON CORE ISSUES: RELEVANCE + CONSTRAINT TRACKING

- GSM-NoOp shows: the model compiles an irrelevant clause into an operation.
- Planning critique failures show: the model can't reliably detect violated preconditions or missing constraints.
- Same underlying weakness:
  - mapping language to formal actions/operations is brittle
  - constraint maintenance through steps is unreliable
  - introspective text does not equal semantic checking

IMPLICATIONS FOR US

- Stop treating "reflection prompts" as a safety layer.
- Treat reliability as **architecture**, not rhetoric:
  - separate interpretation from execution
  - use explicit constraint representations
  - use executable verifiers (calculators, solvers, simulators, rule engines)
  - use LLMs for proposal generation, hypothesis expansion, explanation, and retrieval
- Evaluation should be distributional:
  - perturb numbers/names
  - add irrelevant-but-plausible facts
  - increase compositional depth
  - measure variance and tail risk, not just mean accuracy
- For our domains (triage, compliance, …): the "NoOp" analogue is ubiquitous; build the verifier.